山东大学 计算机科学与技术 学院

自然语言处理 课程实验报告

学号：202200130077	姓名：薛润泽	班级：智能 22

实验题目： 实验 5 词向量

实验内容：

1.One-hot 编码实验

使用 One-hot 独热编码技术将 x=['体育', '军事', '娱乐', '教育', '文化', '时尚', '科技', '财经']处理为向量形式。并思考 one-hot 编码的缺点是什么。

2.基于 Python 和 gensim 框架实现 Word2vec 在 Wikipedia 语料集上面的应用，并且获取词的词向量以及寻找相近词

实验过程中遇到和解决的问题：

（记录实验过程中遇到的问题，以及解决过程和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

1、Wikipedia 语料库比较大，需要在语料库中选择一部分子集作为训练集。助教已经给出了示例代码帮助解决这个问题。

2、使用 Word2vec 时训练过程比较慢，需要耐心等待。

实验步骤：

1、One-hot 编码

（1）使用 sklearn 的库函数帮助编码：

```
from sklearn.preprocessing import OneHotEncoder
```

（2）one-hot 编码的缺点：one-hot 编码是一种稀疏编码，当原始分类特征具有大量唯一值时会造成维度灾难。同时因为是稀疏编码，所以无法表达类别之间的关系。比如无法得到’king’+’woman’ = ‘queen’的规律。

2、使用 Word2vec 获取词的词向量以及寻找相近词

（1）Word2vec 与 one-hot 编码有很大不同的是 word2vec 编码的结果是稠密编码，相比与 one-hot 的稀疏编码不会造成维度灾难。同时 word2vec 可以得到类别之间的规律，比如可以得到如‘king’ + ‘woman’ = ‘queen’之类的规律。

（2）首先对 Wikicorpus 数据进行预处理，进行数据提取

（3）然后对提取后的数据只保留中文，并将繁体中文转化为简体中文

（4）数据处理完成后，调用 Word2Vec 模型进行模型训练，然后进行相近词测试。

实验结果：

One-hot 编码结果如下：

```
(word_vector) D:\大学\大三下\自然语言处理\实验\exp5>python ex5_1.py
体育: [1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
军事: [0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
娱乐: [0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0]
教育: [0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0]
文化: [0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0]
时尚: [0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0]
科技: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0]
财经: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]
```

Word2Vec 进行相近词寻找结果如下，分别寻找“中国”，“自然语言”，“美丽动人”，“助教”等四组词语。

寻找“中国”结果如下：

```
over clean_and _cut
Word2Vec 模型训练并保存完成!
“中国”词向量:  [ 3.5921235   2.5852082   0.6306553   2.3789625  -0.35387072  0.18398826
 -1.030431   -1.2385019  -2.4766943  -1.6767688  -0.4021968   0.36076236
 -0.94411117  1.0129955   2.4652467   2.9359226   0.45542416  0.48069996
  1.6311685  -2.785524   -0.03284286  0.47117138 -5.622671    1.184372
 -0.68540144 -2.388732   -0.3508407   0.24472857  3.532194   -3.5228531
 -1.2897367  -0.20892683  0.98334694  0.34725812  0.6673917   0.02523162
 -1.1473942   0.80019414 -0.8732458   2.8625436   3.6169014  -1.5009446
 -1.2058685   0.05570815 -4.5982985   1.0272372  -2.7321367   0.8691701
 -0.11981159 -2.24209    -1.116979   -1.210277    0.35763717  0.749944
  1.3300314   0.9434801   2.1596112  -0.06282991  1.1597925  -1.2262261
 -2.3156388   0.19115397 -0.6579001  -0.26489663  0.6116634   0.4675892
 -0.99606156 -0.96220076 -2.9671311  -1.4283794  -2.7980165   0.26633117
  0.27404013 -1.303746   -0.2410521  -0.15292221  0.7720919   0.5279732
 -1.192283   -0.9136857  -2.1494477   1.2654465  -0.54737884 -0.8281638
 -1.9109125   0.73471206 -1.167655   -0.7213393  -1.5779432   0.5457066
  0.00588016  0.8746508  -2.7889903   1.805872   -2.2682106  -1.5177798
  0.7449391  -0.38257405  1.3967637   0.38366506]
与“中国”相近的词:
寻奇 : 0.6204
我国 : 0.5618
北京 : 0.5589
埃列布 : 0.5573
东亚 : 0.5358
魔塔 : 0.5319
亚洲 : 0.5286
海监 : 0.5159
欧洲 : 0.5051
中国政府 : 0.5011
```

寻找“自然语言”结果如下：

```
(word_vector) D:\大学\大三下\自然语言处理\实验\exp5>python ex5_2.py
over extract_wiki
over clean_and _cut
“自然语言”词向量:  [ 0.6346461   0.740591   -0.24739619 -0.25455323 -0.0452058  -0.38773125
  0.39813274 -0.24429913  0.05804984 -0.6265441  -0.21258064  0.4663054
 -0.14998667  0.24260382 -0.16348988 -0.00790554 -0.26444763  0.56997126
  0.5298261   0.7022503  -0.7093535  -0.46369362 -0.14895076 -1.6410452
 -0.17358059 -0.7777736  -0.32510963 -0.5688612  -0.17462541 -0.05772085
  0.16343929 -0.48574167 -0.3863447  -0.0500554  -1.2729253   0.97055525
  0.35877398 -0.06941435  0.2212011   0.05571253  0.5907612  -0.25061312
 -0.7283637   0.5148114   0.28215232 -0.21523294 -0.6533794   0.8373004
 -0.37146822 -0.00885671  1.1295378   0.19087301  0.5667922  -0.68237907
 -0.602541   -0.28474334  0.57098556  0.20666404  0.22245157  0.6874197
  0.0507108  -0.07668415  0.1821733  -0.4790411  -0.04249347  0.09078683
  0.41785336 -0.2222545  -0.5247964  -0.59742457 -0.2527155   0.49889198
  0.13724469  0.00998982 -1.2948925   0.64406073  0.18214682  0.27275643
 -0.50605637  0.09592541  0.01555081  0.79920906 -0.45211655  0.26334643
  0.19715632 -0.35800168 -1.1155535  -0.14720058  0.30258283 -0.54336166
 -0.28784093  0.5766692   0.6574102   0.94237787 -0.14069     0.5478769
  1.0421417   0.25856465  0.19200575 -0.7067453 ]
与“自然语言”相近的词:
语义学 : 0.8085
正则表达式 : 0.8000
语义 : 0.7984
方法学 : 0.7834
神经网络 : 0.7795
形式语言 : 0.7723
数据挖掘 : 0.7709
密码学 : 0.7670
机器翻译 : 0.7662
语料库 : 0.7568
```

寻找“美丽动人”结果如下：

```
(word_vector) D:\大学\大三下\自然语言处理\实验\exp5>python ex5_2.py
over extract_wiki
over clean_and _cut
美丽动人词向量:  [ 0.01376566 -0.02916362  0.07913388  0.0503657  -0.03678629 -0.06398474
  0.06453927  0.04756453 -0.12862942 -0.07607964 -0.0627434  -0.07247889
  0.02063021  0.14904132 -0.00767548  0.03972341 -0.05222875  0.0244686
 -0.06479369 -0.19923823 -0.08255778 -0.02392269  0.0400568  -0.04250323
 -0.03018425 -0.04730339 -0.02447911 -0.04904418 -0.05779311  0.0741572
  0.10817026  0.01789481 -0.13941278 -0.15875825  0.02540188  0.00247975
 -0.01111178  0.04691849  0.04170683 -0.19387195  0.00586376 -0.10449117
 -0.18905677 -0.02643373  0.07160006  0.00264095 -0.02732852  0.04015877
  0.08605859  0.02766177 -0.00579647 -0.0232417   0.062272    0.00589657
  0.00765512 -0.00941521  0.09179767  0.03605121  0.00946936  0.00380084
 -0.07988679  0.03398016  0.00345553  0.00184839 -0.13028607  0.20048022
 -0.00145056 -0.0135936  -0.09344675  0.03464542  0.00858494  0.17735294
  0.13513957 -0.05651332  0.07410458  0.00414733  0.01198558 -0.12230296
 -0.04807727 -0.04560108  0.10775438 -0.02953612 -0.08448951 -0.0975607
  0.04035316 -0.11720838 -0.04741337  0.03100634  0.07004203  0.12435067
  0.15643324  0.01402351  0.07436357 -0.02396559  0.03329922  0.06541406
  0.1727416  -0.15592411  0.00812465  0.0328586 ]
与美丽动人相近的词:
纤瘦 : 0.8116
乌黑 : 0.8079
美腿 : 0.7944
风度翩翩 : 0.7934
姣好 : 0.7905
偏白 : 0.7844
腰束 : 0.7839
黑白分明 : 0.7827
娇艳 : 0.7814
红润 : 0.7783
```

寻找“助教”结果如下：

```
(word_vector) D:\大学\大三下\自然语言处理\实验\exp5>python ex5_2.py
over extract_wiki
over clean_and _cut
助教词向量:  [-0.59596044  0.02927952 -0.25577056  0.4114716  -0.9449691   0.00342556
  0.2728081   1.9060469   1.8096355  -0.9401802   0.26033577 -0.7048709
 -1.2125928   0.9041917   0.5309427  -1.4582822  -0.00410653  0.67452997
  1.4737504  -0.13151926 -0.4552728  -0.13122422  1.3273637  -0.576679
 -1.3077079  -1.1731973  -0.77348506  0.3420138  -0.43002596 -0.8427925
 -0.49723136  1.3693798   0.6067435  -0.09125026 -0.03843416 -0.36889377
  0.69387287 -0.12510575 -1.6412284   0.7968379  -0.15312988 -0.55901873
  1.1810791   0.47777227  0.40805018  0.53852344  1.3528006   0.20843263
  0.35692862 -0.16638981 -0.04047286  1.2560865   1.6109017  -0.10612758
 -0.96384877 -1.114536    1.8859893   0.7376819  -0.29450324 -0.8012543
  0.6493501   1.6578734   0.8330519  -0.9989176   1.258466    0.3208764
 -1.5417912   1.9475131   0.42938006  0.27568588  0.5184939  -0.34783384
  1.8001816   0.6973775   0.0874301   0.34984043  0.68722266  0.5026173
 -0.6777255   0.3387586   0.5039998   0.05836676 -0.41606593 -0.3659891
  0.78335905 -0.7394356  -1.087104    0.46342906  0.83593094 -0.7332775
  0.79970586  0.40723577  0.17572814 -0.33567446  0.19273275  1.5742078
  0.08291399 -1.2541035   1.0760016   0.8596249 ]
与助教相近的词:
教务长 : 0.7857
留校 : 0.7700
讲师 : 0.7676
教务主任 : 0.7602
教职 : 0.7536
应聘 : 0.7515
副校长 : 0.7423
副教授 : 0.7343
客座教授 : 0.7316
物理系 : 0.7148
```

实验总结：

相比与 One-hot 独热编码，使用 Word2Vec 编码能够获取类别之间的关系，且降低了维度爆炸风险，更好用。

附源代码

#One-hot 独热编码

```python
from sklearn.preprocessing import OneHotEncoder
import numpy as np

def onehot_encode(x):
    x = np.array(x).reshape(-1, 1)
    encoder = OneHotEncoder()
    onehot_sparse = encoder.fit_transform(x)
    onehot = onehot_sparse.toarray()
    for word, vec in zip(x, onehot):
        print(f"{word[0]}: {vec.tolist()}")


x = ['体育', '军事', '娱乐', '教育', '文化', '时尚', '科技', '财经']
```

```
onehot_encode(x)



#Word2Vec
import os
import re
import jieba
import zhconv
from gensim.corpora import WikiCorpus
from gensim.models import Word2Vec
import multiprocessing
from gensim.models import word2vec

def extract_wiki(input_file, output_file, limit=50000):
    wiki_corpus = WikiCorpus(input_file, dictionary={})
    with open(output_file, 'w', encoding='utf-8') as f:
        i = 0
        for text in wiki_corpus.get_texts():
            f.write(' '.join(text) + '\n')
            i += 1
            if i % 10000 == 0:
                print(f'已处理 {i} 篇文章')
            if i >= limit:
                break


def clean_and_cut(input_file, output_file):
    with open(input_file, 'r', encoding='utf-8') as fin, \
         open(output_file, 'w', encoding='utf-8') as fout:
        for line in fin:
            line = zhconv.convert(line, 'zh-cn')
            line = re.sub(r'[^\u4e00-\u9fa5]', '', line)
            words = jieba.cut(line)
            fout.write(' '.join(words) + '\n')


def train_word2vec(corpus_file, model_path):
    sentences = list(word2vec.LineSentence(corpus_file))
    model = Word2Vec(
        vector_size=100,
        window=5,
        min_count=5,
        workers=multiprocessing.cpu_count()
```

```
    )
    model.build_vocab(sentences)
    model.train(sentences, total_examples=model.corpus_count,
epochs=model.epochs)
    model.save(model_path)
    return model


def test_model(model, aim_word): #aim_word type=str
    print(aim_word+"词向量：", model.wv[aim_word])
    print("与"+aim_word+"相近的词：")
    for word, score in model.wv.most_similar(aim_word, topn=10):
        print(f"{word} : {score:.4f}")
if __name__ == '__main__':
    input_file_name = 'zhwiki-latest-pages-articles.xml.bz2'
    output_file_name = 'corpus_cn.txt'
    clean_corpus_file = 'corpus_cut.txt'
    model_path = 'wiki_word2vec.model'
    if not os.path.exists(output_file_name):
        extract_wiki(input_file_name, output_file_name)
    print('over extract_wiki')
    if not os.path.exists(clean_corpus_file):
        clean_and_cut(output_file_name, clean_corpus_file)
    print('over clean_and _cut')
    if not os.path.exists(model_path):
        model = train_word2vec(clean_corpus_file, model_path)
    else:
        model = Word2Vec.load(model_path)
    test_model(model,"助教")
```